

# Hybrid methods for optimizing a linear fractional function over the integer efficient set: what each hybridization is worth, measured

## Abstract

Optimizing a linear fractional function  $\Phi$  over the efficient set of a multi-objective integer program is usually done by repeatedly truncating the *decision* space with Sylva–Crema cuts. Each cut states a disjunction, so it costs  $p$  binary variables and  $p + 1$  big- $M$  rows, and the sub-problem solved at every iteration is strictly larger than the one before it. We first move the unexplored part of the problem into *criterion* space, as a list of boxes: the truncation becomes a box subtraction, every sub-problem is the original feasible set plus a handful of ordinary linear rows, and the model never grows. On 35 instances this is faster on 34, by  $5.05\times$  overall and  $9.01\times$  on the heaviest, with the margin widening in  $p$ .

We then hybridize that exact search three ways and report what each is worth. Handing it the unfinished work of the published method (exact–exact) *ties* it. Seeding it from a Pareto local search with an archive (exact–metaheuristic) gives  $1.25\times$  against a measured ceiling of  $1.42\times$ . Seeding it from the augmented weighted Tchebychev program — the scalarization the surrounding literature generates efficient points with — *loses* to not seeding at all, at  $0.71\times$ .

The last result is the informative one, and it does not say the scalarization is weak. As a generator it is strictly stronger than a weighted sum, reaching unsupported efficient points no positive weighted sum can reach. It is a poor *seed* because it steers in criterion space relative to the ideal point and never looks at  $\Phi$ . Symmetrically, the same free optimum that saves the box search a third of its work saves the decision-space method *zero* iterations. What a heuristic incumbent is worth is a property of the exact method it is attached to, not of the heuristic; we measure this in both directions. All arithmetic is exact and rational, every answer is proved optimal and cross-checked against independent references, and the negative results and two corrected measurement errors are reported alongside the wins.

## Contents

|          |                                                                     |          |
|----------|---------------------------------------------------------------------|----------|
| <b>1</b> | <b>Problem</b>                                                      | <b>2</b> |
| <b>2</b> | <b>Where the cost of the decision-space method comes from</b>       | <b>2</b> |
| <b>3</b> | <b>The same truncation in criterion space</b>                       | <b>3</b> |
| 3.1      | How a box is written, and why not with numbers . . . . .            | 3        |
| 3.2      | The split . . . . .                                                 | 3        |
| 3.3      | Bounding, and why it is what makes seeding possible . . . . .       | 4        |
| <b>4</b> | <b>Two obstacles specific to a fractional objective</b>             | <b>4</b> |
| <b>5</b> | <b>The algorithm</b>                                                | <b>4</b> |
| <b>6</b> | <b>Computational results: decision space versus criterion space</b> | <b>5</b> |
| <b>7</b> | <b>First hybridization: exact–exact</b>                             | <b>6</b> |

|           |                                                                            |           |
|-----------|----------------------------------------------------------------------------|-----------|
| <b>8</b>  | <b>Second hybridization: exact–metaheuristic</b>                           | <b>6</b>  |
| 8.1       | The heuristic . . . . .                                                    | 6         |
| 8.2       | Where correctness lives . . . . .                                          | 7         |
| 8.3       | What it costs, and what it is worth . . . . .                              | 7         |
| <b>9</b>  | <b>The asymmetry: a seed is worth what the exact method can do with it</b> | <b>7</b>  |
| <b>10</b> | <b>Third hybridization: the Tchebychev reference</b>                       | <b>8</b>  |
| 10.1      | The program . . . . .                                                      | 8         |
| 10.2      | As a generator it is strictly stronger than a weighted sum . . . . .       | 8         |
| 10.3      | As a seed it loses to not seeding at all . . . . .                         | 9         |
| 10.4      | The two generators compared directly . . . . .                             | 9         |
| <b>11</b> | <b>Two measurement errors, and what they cost</b>                          | <b>10</b> |
| <b>12</b> | <b>Summary of what was and was not gained</b>                              | <b>10</b> |
| <b>13</b> | <b>Prior work, and what remains to be checked</b>                          | <b>10</b> |
| <b>14</b> | <b>Reproducibility</b>                                                     | <b>11</b> |

## 1 Problem

Let

$$(P_D) \quad \text{“max”} \quad Z_k(x) = \frac{c_k^\top x + a_k}{d_k^\top x + b_k}, \quad k = 1, \dots, p, \quad x \in D = \{x \in \mathbb{Z}_+^n : Ax \leq b\}, \quad (1)$$

with  $D$  non-empty and bounded and every denominator  $D_k(x) = d_k^\top x + b_k$  strictly positive on  $D$ . A point  $x \in D$  is *efficient* when no  $y \in D$  satisfies  $Z(y) \geq Z(x)$  with at least one strict inequality;  $E(P_D)$  denotes the set of efficient points. Linear criteria are the degenerate case  $d_k = 0, b_k = 1$ .

Given the decision maker’s criterion  $\Phi(x) = (U^\top x + \alpha)/(V^\top x + \beta)$ , the problem is

$$(P_E) \quad \max \Phi(x) \quad \text{s.t.} \quad x \in E(P_D). \quad (2)$$

$E(P_D)$  is a union of faces of  $D$ : non-convex, with no explicit description. The point of any method for  $(P_E)$  is to reach the global optimum *without enumerating*  $E(P_D)$ .

An efficient point is *supported* when it maximizes some strictly positive weighted sum  $w^\top Z$  over  $D$  — equivalently, when its criterion vector lies on the convex hull of the criterion image. The distinction matters throughout Section 10: in an integer program unsupported efficient points are common, and no weighted sum reaches any of them.

## 2 Where the cost of the decision-space method comes from

One iteration of the published method maximizes  $\Phi$  over a truncated region  $D_l$ , tests the maximizer  $x_l$  for efficiency, banks the best  $\Phi$  on the criterion slice about to be destroyed, and then truncates:

$$\text{delete} \quad \{x : Z(x) \leq Z(\tilde{x})\}. \quad (3)$$

Keeping the complement means keeping the  $x$  for which *some* criterion strictly improves on  $\tilde{x}$ . That is a disjunction, and a disjunction is not a system of linear inequalities. It is modelled

with one binary per criterion,

$$e_k(x; \tilde{x}) \geq 1 \cdot y_k + M_k(1 - y_k), \quad k = 1, \dots, p, \quad \sum_{k=1}^p y_k \geq 1, \quad (4)$$

where  $M_k$  is a lower bound on  $e_k$  over the region, so  $y_k = 0$  leaves its row vacuous and  $y_k = 1$  forces a strict improvement.

**Every cut therefore adds  $p$  binaries and  $p+1$  rows.** On the heaviest instance we ship, eleven cuts leave 33 binaries and 44 rows on top of  $D$ , and the late sub-problems cost far more than the early ones. This is the whole of the method's cost profile: the number of iterations is modest, but each one is solved on a strictly larger mixed-integer program than the last.

This observation fixes the cost model used for every comparison in this note:

$$\text{total cost} \approx (\text{number of sub-problems solved}) \times (\text{size of a sub-problem}). \quad (5)$$

The two factors are attacked by different ideas. Section 3 removes the growth of the second. Sections 8 and 10 attack the first, and the measurements there are only interpretable because the second has already been made constant.

### 3 The same truncation in criterion space

Equation (3) deletes a *box* from  $\mathbb{R}^p$ . If the unexplored part of the problem is carried as a list of boxes rather than as a region of  $\mathbb{Z}^n$ , the truncation is a rewrite of that list, and each sub-problem is

$$\max \Phi(x) \quad \text{s.t.} \quad x \in D, \quad \text{the box's rows}, \quad (6)$$

which is  $D$  plus at most  $2p$  ordinary linear rows. No binary variable appears, and **the model never grows**: a box deep in the search carries a few more rows than a shallow one, and that is all.

#### 3.1 How a box is written, and why not with numbers

The obvious encoding,  $l_k \leq Z_k(x) \leq u_k$ , fails. In the general case  $Z_k$  is a ratio, so its values are rational, and the exact “+1” step that the split below needs would be lost.

Write the box with the linear form of the paper's Theorem 4 instead:

$$e_k(x; a) = D_k(a) D_k(x) (Z_k(x) - Z_k(a)) = (D_k(a)c_k - N_k(a)d_k)^\top x + (D_k(a)a_k - N_k(a)b_k). \quad (7)$$

Both denominators being positive,  $e_k$  carries the sign of  $Z_k(x) - Z_k(a)$  while being *linear in  $x$* ; with integer data it is integer-valued on integer points. Hence, exactly and with no minimal step to estimate,

$$Z_k(x) > Z_k(a) \iff e_k(x; a) \geq 1, \quad Z_k(x) \leq Z_k(a) \iff e_k(x; a) \leq 0. \quad (8)$$

Linear and fractional criteria therefore go through the same code, and on a linear criterion  $e_k$  collapses to  $c_k^\top x - c_k^\top a$ .

#### 3.2 The split

**Proposition 1.** *Let  $B$  be a box and  $a \in D$ . For  $k = 1, \dots, p$  put*

$$B_k = B \cap \{x : e_k(x; a) \geq 1\} \cap \bigcap_{j < k} \{x : e_j(x; a) \leq 0\}. \quad (9)$$

*Then  $B_1, \dots, B_p$  are pairwise disjoint and  $\bigcup_k B_k = B \setminus \{x : Z(x) \leq Z(a)\}$ .*

*Proof.* Let  $x \in B$  with  $Z(x) \not\leq Z(a)$ . The set of indices at which  $x$  strictly beats  $Z(a)$  is non-empty; let  $k$  be its smallest element. By (8),  $e_k(x; a) \geq 1$ , and  $e_j(x; a) \leq 0$  for every  $j < k$ , so  $x \in B_k$ . Conversely a point of  $B_k$  has  $e_k \geq 1$ , hence  $Z(x) \not\leq Z(a)$ . Disjointness: membership of  $B_k$  forces  $k$  to be exactly that smallest index, and a point has only one.  $\square$

Each of the  $p$  children is the parent's row list plus at most  $k$  further rows, all of them ordinary linear inequalities in  $x$ .

### 3.3 Bounding, and why it is what makes seeding possible

The sub-problem (6) maximizes  $\Phi$  over a *superset* of the efficient points whose criterion vector lies in the box, so its value bounds all of them. A box whose bound does not beat the incumbent is therefore discarded *whole* and never split. Children inherit their parent's value as a bound, which makes the open list a certified upper bound on everything still unfound; the method is *anytime* in the same sense, and for the same reason, as the decision-space one.

This is the single structural fact that Sections 8 and 10 rest on. A better incumbent does not merely stop the search sooner; it deletes subtrees. The decision-space method has no such mechanism, which is measured in Section 9 and is the reason the two methods respond so differently to the very same incumbent.

## 4 Two obstacles specific to a fractional objective

Criterion-space search is a known family for optimizing a *linear* function over the efficient set. It does not transfer to  $(P_E)$  by inspection, for two reasons.

**Remark 1** ( $\Phi$  is a function of  $x$ , not of  $Z(x)$ ). *Two feasible points with the same criterion vector may have different values of  $\Phi$ . Enumerating non-dominated vectors and evaluating  $\Phi$  on a representative of each is therefore wrong. Inside every box a genuine fractional integer program must be solved, and before cutting on an efficient centre  $\tilde{x}$  the sub-problem  $Q(\tilde{x}) = \max\{\Phi(x) : x \in D, Z(x) = Z(\tilde{x})\}$  must be solved first, since the whole slice  $\{Z = Z(\tilde{x})\}$  is efficient and is about to be removed.*

**Remark 2** (Box boundaries must be integer-valued). *As argued above, numeric bounds on  $Z_k$  are rational in the fractional case. Writing the boundaries through  $e_k$  restores an exact integral threshold and, as a by-product, makes fractional criteria work with no special case.*

## 5 The algorithm

Termination: every pass removes at least the criterion vector  $Z(a)$ , and the non-dominated set of a bounded integer program is finite.

Line 2 is the whole of the hybridization interface. Everything in Sections 8 and 10 is a different way of filling it, and the correctness requirement is stated once, in Proposition 2 below: the seed must be *efficient*, not merely good.

**Proposition 2** (Soundness of an arbitrary seed). *If the seed  $x_0$  in line 2 is efficient, Algorithm 1 returns the same  $(x_{\text{opt}}, \Phi_{\text{opt}})$  as the unseeded run, and  $\Phi_{\text{opt}}$  is the global optimum of  $(P_E)$ . If  $x_0$  is merely feasible, the algorithm may return a strictly suboptimal value.*

*Proof.*  $\Phi_{\text{opt}}$  is used only as a cutoff, and a box is discarded when its bound fails to *beat* it. If  $x_0 \in E(P_D)$  then  $\Phi(x_0) \leq \max_{x \in E(P_D)} \Phi(x)$ , so no box containing an optimal criterion vector is ever discarded, and the incumbent is a feasible value of  $(P_E)$  throughout; the two runs therefore agree. If  $x_0 \notin E(P_D)$  then  $\Phi(x_0)$  may exceed the optimum of  $(P_E)$  — a dominated point can score arbitrarily well on  $\Phi$  — and the box containing the true optimum is discarded by the very first bound comparison.  $\square$

---

**Algorithm 1** Criterion-space search for  $(P_E)$ , optionally seeded

---

```
1:  $L \leftarrow \{\mathbb{R}^p\}$ 
2:  $(x_{\text{opt}}, \Phi_{\text{opt}}) \leftarrow$  a verified efficient seed and its value, or  $(\emptyset, -\infty)$ 
3: while  $L \neq \emptyset$  do
4:   remove from  $L$  a box  $B$  of largest inherited bound
5:   solve (6) on  $B$  with cutoff  $\Phi_{\text{opt}}$ 
6:   if infeasible, or its value  $\leq \Phi_{\text{opt}}$  then
7:     continue  $\triangleright B$  is discarded whole
8:   end if
9:   let  $x_B$  be its maximizer
10:  if  $x_B$  is efficient then
11:     $\Phi_{\text{opt}} \leftarrow \Phi(x_B)$ ;  $x_{\text{opt}} \leftarrow x_B$ ;  $a \leftarrow x_B$ 
     $\triangleright x_B$  already maximizes  $\Phi$  on its whole slice, so no  $Q$  is owed
12:  else
13:     $a \leftarrow$  an efficient point dominating  $x_B$ 
14:    solve  $Q(a)$  and update  $(\Phi_{\text{opt}}, x_{\text{opt}})$ 
15:  end if
16:  add to  $L$  the  $p$  children of  $B$  given by (9)
17: end while
18: return  $(x_{\text{opt}}, \Phi_{\text{opt}})$ , proved optimal
```

---

This is why every seed in this note is *verified* before it is used, and why that verification, not the heuristic, turns out to be the cost of Section 8.

## 6 Computational results: decision space versus criterion space

All figures come from an exact rational implementation with no floating-point arithmetic anywhere, so every comparison below is between two exactly solved problems. Both methods were run on the same instances in the same process.

Table 1: Decision space versus criterion space,  $p = 3$ .

| instance                | decision space | criterion space | ratio |
|-------------------------|----------------|-----------------|-------|
| 35 instances, total     | 40.65 s        | 8.06 s          | 5.05× |
| heaviest $n = 10$       | 23.47 s        | 2.61 s          | 9.01× |
| $n = 8$ , hardest seed  | 2.97 s         | 0.45 s          | 6.56× |
| $n = 10$ , easiest seed | 0.36 s         | 0.41 s          | 0.86× |

The box search is faster on 34 of the 35, and the single regression is the instance the decision-space method settles in three iterations, where paying 13 boxes is a net loss. Every answer agrees with the decision-space method and with an independent scan, and both are proved optimal everywhere.

Two forces grow with  $p$  and pull against each other: the split produces  $p$  children, so the list branches wider; but the non-dominated set grows too, so the decision-space method needs more cuts, and each cut costs it  $p$  binaries and  $p+1$  rows. The measurement settles which dominates, and it settles it against the prediction we made before running it. Branching multiplies a *count* of cheap, fixed-size models; the binaries multiply the *cost of one model*, and they accumulate. The worst case observed for the published method was  $p = 5$  on six variables: 110.95 s against 0.64 s, a factor of 173.

Table 2: The margin against the number of criteria,  $n = 6$ , four instances each. All twenty agreed and were proved optimal on both sides.

| $p$ | decision space | criterion space | ratio |
|-----|----------------|-----------------|-------|
| 2   | 0.61 s         | 0.35 s          | 1.74× |
| 3   | 1.02 s         | 0.34 s          | 2.99× |
| 4   | 4.21 s         | 0.61 s          | 6.92× |
| 5   | 112.77 s       | 1.09 s          | 103×  |
| 6   | 26.15 s        | 1.08 s          | 24.3× |

**On the magnitudes.** The direction is solid: the per- $p$  median ratios are 1.35, 3.2, 2.1, 12.6, 15.8, and every instance agreed and was proved optimal. The totals are not stable estimates — four instances per  $p$  is few, and 110.95 of the 112.77 seconds at  $p = 5$  come from a single instance. The tables report what was measured, not expected values.

## 7 First hybridization: exact–exact

Since the published method closes in one iteration whenever the maximizer of  $\Phi$  over  $D$  happens to be efficient, a hybrid suggests itself: run its loop, and if it has not closed after  $s$  iterations, hand the unfinished work to the box search. The handover is not a restart. Replaying each cut as a box subtraction (Proposition 1 applied to every box of the current list) reproduces the region the first phase reached, and the incumbent, being attained at a known efficient point, is a valid seed by Proposition 2.

Measured on 24 instances against the better of the two pure methods on each:

|                      | total    | within 15% of the better |
|----------------------|----------|--------------------------|
| $s = 3$              | 13.14 s  | 4 of 24                  |
| $s = 1$              | 8.47 s   | 23 of 24                 |
| pure box search      | 8.11 s   | —                        |
| the published method | 268.61 s | —                        |

We report this as a negative result. At  $s = 1$  the hybrid *ties* the pure box search — 8.47 s against 8.11 s here, and 1.47 s against 1.54 s on a family built so that  $\Phi$  is easy; two differences of about 4% pointing opposite ways. The criterion-space search is what matters; this hybrid earns its place only as insurance for the one-iteration case, not as a third method that beats both. The first default we chose,  $s = 3$ , was measurably wrong and is reported here because it was: 13.14 s against 8.47 s.

## 8 Second hybridization: exact–metaheuristic

By Proposition 2 the box search will accept any efficient point as a seed, and by Section 3.3 a good one deletes subtrees. That makes a metaheuristic the natural first phase — it is cheap and it is guided by  $\Phi$ , which is exactly the quantity the cutoff compares against.

### 8.1 The heuristic

A Pareto local search maintains an archive of mutually non-dominated points. From several random maximal starting points it walks the neighbourhood of each archive member, admitting any neighbour the archive does not dominate. The archive *is* the search: restarting from fresh

random points between walks throws it away, and four times the effort did not compensate for that (obtaining 0.360, 0.280 and 0.458 of the optimum on three instances where the archive-driven version obtained the optimum).

The neighbourhood needs *swap* moves ( $-1$  on one coordinate,  $+1$  on another), not only unit steps. From a maximal point a unit step up is infeasible, and a unit step down lowers every criterion with non-negative coefficients, so the archive refuses it and the walk dies immediately: three archive points and an incumbent at 13% of the optimum. With swaps the archive holds 28–69 points and the incumbent reaches 60–100%.

## 8.2 Where correctness lives

The archive is filtered by dominance *among the points it has seen*, which is not the exact efficiency test. Proposition 2 makes that gap fatal if ignored, so nothing crosses unverified: the best archive member is put through the exact test, and if it fails, it is walked to an efficient point dominating it and that point’s value is used instead. The heuristic decides *where to look*; it never decides what is true.

The gap is not hypothetical. Over a wide sweep, one archive member in roughly 2000 turned out not to be efficient — reproducibly, a concrete case being a four-variable instance whose archive retains  $(2, 1, 1, 1)$ . Rare is not never.

## 8.3 What it costs, and what it is worth

The cost is *not* the walk, which takes 0.01–0.08 s and is barely sensitive to its budget. We assumed the opposite and optimized the walk first; the timings then showed the total *falling as the walk’s budget rose*, because a poor archive yields candidates needing longer repair chains, each one an integer program. The cost is the verification — the price of Proposition 2. Examining the single best archive member gave exactly the same incumbent as examining four on every instance tried, at a quarter to a half of the cost.

Table 3: Exact–metaheuristic hybrid, 18 instances. Same optimum everywhere, all proved optimal.

| instance            | exact only        | metaheuristic + exact | ratio |
|---------------------|-------------------|-----------------------|-------|
| total, 18 instances | 9.50 s            | 7.60 s                | 1.25× |
| hardest $n = 10$    | 49 boxes, 3.02 s  | 34 boxes, 1.94 s      | 1.56× |
| $n = 8$ , seed 0    | 16 boxes, 0.53 s  | 10 boxes, 0.34 s      | 1.53× |
| $p = 7$ , seed 1    | 197 boxes, 1.36 s | 120 boxes, 0.95 s     | 1.43× |
| $p = 5$ , seed 1    | 6 boxes, 0.02 s   | 6 boxes, 0.03 s       | 0.56× |

It wins where the search is expensive and loses where there was nothing to win, which is what a nearly fixed first-phase cost predicts. The *ceiling*, measured by handing the search the true optimum for free, is 1.42×; at 1.25× the heuristic realizes 88% of it.

## 9 The asymmetry: a seed is worth what the exact method can do with it

The ceiling experiment can be run on the published method too, and this is the comparison we consider the most transferable result in this note.

The reason is structural. What the decision-space method cuts at iteration  $l$  is decided by the efficiency test applied to  $x_l$ , through (3);  $\Phi_{\text{opt}}$  enters the sub-problem only as a cutoff inside a branch-and-bound that would have found the same  $x_l$  anyway. A box, by contrast, is compared

Table 4: The *same* free optimum, handed to each exact method at iteration 1.

| exact method               | effect of a free optimal incumbent | mechanism             |
|----------------------------|------------------------------------|-----------------------|
| criterion-space box search | 1.42×, up to 52% fewer boxes       | boxes discarded whole |
| decision-space cut method  | <b>zero</b> iterations saved       | none                  |

against the incumbent *before* it is ever split, and failing that comparison deletes it together with every descendant it would have had.

So “hybridize an exact method with a metaheuristic” is not one question with one answer. The heuristic is identical in both rows above; what differs is whether the exact method has a mechanism through which a lower bound can remove work. Before building such a hybrid, the mechanism is what should be checked.

## 10 Third hybridization: the Tchebychev reference

The surrounding literature generates efficient points with the augmented weighted Tchebychev program, so a fair evaluation of the two hybrids above has to be made against it and not only against a weighted sum. We therefore implement it and measure it, rather than cite it.

### 10.1 The program

With  $z^*$  the ideal point,  $z_k^* = \max\{Z_k(x) : x \in D\}$ , weights  $w > 0$  and an augmentation  $\rho > 0$ :

$$\min_{x \in D} \max_k w_k(z_k^* - Z_k(x)) + \rho \sum_{k=1}^p (z_k^* - Z_k(x)), \quad (10)$$

which linearizes with a single continuous variable  $\lambda$  subject to  $\lambda \geq w_k(z_k^* - Z_k(x))$  for each  $k$ . Each weight vector therefore costs one integer program on a model with  $n + 1$  variables and  $p$  extra rows, plus  $p$  programs paid once for  $z^*$ .

**Proposition 3.** *For any  $w > 0$  and any  $\rho > 0$ , every optimal solution of (10) is efficient.*

*Proof.* Let  $x^*$  be optimal and suppose some  $y \in D$  dominated it, so  $Z(y) \geq Z(x^*)$  with at least one strict inequality. Then  $z_k^* - Z_k(y) \leq z_k^* - Z_k(x^*)$  for every  $k$ , so the max term does not increase; and since  $\rho > 0$  and at least one inequality is strict, the sum term *strictly* decreases. The objective at  $y$  would be strictly smaller than at  $x^*$ , contradicting optimality.  $\square$

The augmentation is exactly what makes the second half of that argument available: with  $\rho = 0$  the objective at  $y$  need only be no larger, and a merely *weakly* efficient point can minimize the program. Our implementation therefore refuses  $\rho \leq 0$  and refuses a zero weight rather than returning a point whose status is in doubt. It also refuses fractional criteria, for which  $z_k^* - Z_k(x)$  is not linear and the rows above are not constraints of an integer linear program; the references assume linear criteria as well.

### 10.2 As a generator it is strictly stronger than a weighted sum

A weighted sum maximizes only at supported efficient points. Varying  $w$  in (10) reaches unsupported ones too, and that is what the literature uses it for.

All 540 optima returned were efficient, as Proposition 3 requires. A wider sweep over 40 instances agrees: 216/369 against 135/369, 56 unsupported against 0, and 1080 optima with no inefficient one. On the reference instance of the published paper, three of the seven efficient points are unsupported; the Tchebychev program reaches all three and no strictly positive weighted sum reaches any.

Table 5: Reach from the same grid of 27 weight vectors, 20 random instances.

|                      | efficient points reached | of which unsupported |
|----------------------|--------------------------|----------------------|
| weighted sum         | 66/175                   | <b>0/54</b>          |
| augmented Tchebychev | 112/175                  | 29/54                |

### 10.3 As a seed it loses to not seeding at all

Table 6: Seeding the box search, 18 instances. “Exactly optimal” counts the instances where the seed already attains  $\Phi^*$ .

| seed                   | total                          | seed cost | exactly optimal |
|------------------------|--------------------------------|-----------|-----------------|
| none (pure box search) | 9.64 s                         | —         | —               |
| Pareto local search    | <b>7.97 s</b> (1.21 $\times$ ) | 0.57 s    | 12 of 18        |
| augmented Tchebychev   | 13.50 s (0.71 $\times$ )       | 4.25 s    | 5 of 18         |

Both halves of the loss are structural rather than artefacts of the implementation.

**Cost.** It is  $p + 1$  integer programs on an enlarged model. It is *not* the reference point: computing  $z^*$  took 0.01–0.13 s of the seed against 0.13–0.85 s for the scalarizations themselves, so replacing  $z^*$  by a cheaper reference would not change the verdict.

**Quality.** This is the real finding. Program (10) steers by criterion-space geometry relative to  $z^*$  and **never looks at  $\Phi$** . The local search of Section 8 is guided by  $\Phi$  at every move. A good spread of efficient points is not a good  $\Phi$ , and by Remark 1 it cannot be made into one by post-processing:  $\Phi$  is not a function of  $Z(x)$ , so two points the generator regards as interchangeable may differ arbitrarily in the only quantity the search cares about.

We therefore keep the Tchebychev program as the reference the evaluation needs, and do not wire it into the default hybrid.

### 10.4 The two generators compared directly

Table 7: As generators of a subset of  $E(P_D)$ , 20 instances at  $n = 4$ .

|                      | coverage of $E(P_D)$ | time   | certification                            |
|----------------------|----------------------|--------|------------------------------------------|
| augmented Tchebychev | 64%                  | 2.1 s  | every point efficient by construction    |
| Pareto archive       | 99%                  | 0.04 s | $\approx 1$ member in 2000 not efficient |

Cheap, near-complete and uncertified against expensive, partial and certified. That is the trade the exact–metaheuristic hybrid closes: it takes the cheap, near-complete generator and buys the missing certificate with a single exact test, which is why the verification and not the walk is its dominant cost.

This also delimits the expected result of the thesis — “a subset of efficient solutions generated by hybrid algorithms”. Both columns produce such a subset, and they are not interchangeable: one is a certified partial sample of the front, the other a near-complete sample whose membership is a conjecture until tested.

## 11 Two measurement errors, and what they cost

We report these because the direction of a result is worth less than the knowledge of how it was checked.

**An unsound quality metric.** Seed quality was first scored by the ratio  $\Phi(\text{seed})/\Phi^*$ . That ratio *inverts when  $\Phi^*$  is negative*, which it is on several instances in these families: a strictly worse seed can score above 1. All seed-quality figures in this note use the non-negative gap  $(\Phi^* - \Phi(\text{seed}))/|\Phi^*|$ , which is well defined whatever the sign and is zero exactly when the seed is optimal. No published number changed direction under the correction, but the metric was wrong and one table read 1.286 for a seed that was in fact 29% below the optimum.

**Seed quality is not seed value.** On a looser instance family, the Pareto seed is *exactly optimal on 15 of 24 instances* and still buys only  $1.05\times$ , because that search is not bound-limited: there are few boxes whose bound the incumbent can defeat. Quality is a property of the heuristic; value is a property of the pair. What is stable across all families we tried is the *ordering* — Tchebychev loses to no seed at all ( $0.73\times$ ,  $0.91\times$ ), the Pareto seed ties or wins — not the size of any win. Single speed-up figures quoted without their instance family should be read with this in mind, including ours.

## 12 Summary of what was and was not gained

Table 8: Every hybridization attempted, with its verdict.

| hybridization                                | verdict     | effect                                 |
|----------------------------------------------|-------------|----------------------------------------|
| decision space $\rightarrow$ criterion space | <b>win</b>  | $5.05\times$ , up to $173\times$       |
| exact–exact handover ( $s = 1$ )             | tie         | 8.47 s vs 8.11 s                       |
| exact–exact handover ( $s = 3$ )             | <b>loss</b> | 13.14 s vs 8.47 s                      |
| exact–metaheuristic (Pareto archive)         | <b>win</b>  | $1.25\times$ of a $1.42\times$ ceiling |
| exact–Tchebychev seed                        | <b>loss</b> | $0.71\times$                           |
| Tchebychev as a generator                    | <b>win</b>  | reaches unsupported points             |
| incumbent seeding in decision space          | <b>loss</b> | zero iterations saved                  |

Three of the seven are losses and one is a tie. They are reported at the same length as the wins because they carry the same information: a hybridization pays only where the exact method has a mechanism the first phase can feed, and identifying that mechanism is cheaper than building the hybrid and measuring it afterwards.

## 13 Prior work, and what remains to be checked

Criterion-space search for optimizing a function over the efficient set of a multi-objective integer program is an established family, and the surveys report it as currently more effective than decision-space search. Those methods are for *linear* objectives. The literature on a linear *fractional* objective over an integer efficient set — including the branch-and-cut methods this note compares against, and the extensions to fractional criteria and to quadratic objectives — works in decision space. The augmented weighted Tchebychev program of Section 10 is standard and is used here exactly as the references use it; nothing about it is claimed as new. The combination treated in Section 3 appears not to have been addressed, for the reason given in Remark 1:  $\Phi$  is a function of  $x$  and not of  $Z(x)$ , so the transfer is not immediate.

**This is a statement about what we did not find, not a claim of novelty.** A proper bibliographic review, including the sources behind paywalls, remains to be done before any such claim is made.

## 14 Reproducibility

The implementation is pure Python with no third-party dependency, computes in exact rational arithmetic throughout, and carries 55 tests. Several check properties rather than answers, since a method that silently lost efficient points would still agree with the optimum on favourable instances:

- Proposition 1 directly — for every centre and every feasible point, the point is either removed by the cut and in no child, or kept and in exactly one.
- Proposition 2 operationally — the metaheuristic is checked, over 14 instances and against an independent scan, never to hand over an inefficient point.
- Proposition 3 — every optimum of (10) is checked efficient against exhaustive enumeration, and the refusals ( $\rho \leq 0$ , a zero weight, fractional criteria) are checked to fire.
- The unsupported-reach claim — on the reference instance, the three unsupported efficient points are all reached by (10) while no positive weighted sum reaches any.

Every comparison in this note is reproduced by scripts in the repository, run on every commit by continuous integration, which assert that each seeded run returns the same proved optimum as the unseeded one.